Algorithmie

SAÉ 3.02 - Labyrinthe

Développement efficace

Étudiants :

Victor Bredelle
Antonin Marouze
Romain Harlaut
Angèl Zheng
Baptiste Lavogiez

Enseignant :

Mme Everaere

13 novembre 2025

Table des matières

1 Introduction **3**

2 Investissement dans le projet **4**

3 Structures de données pour les labyrinthes **6**
3.1 Labyrinthes aléatoires 6
3.2 Labyrinthes parfaits 6

4 Algorithmes de génération **7**
4.1 Distance entrée / sortie 7
4.1.1 Algorithme utilisé 7
4.1.2 Pseudo-code 8
4.1.3 Structures de données utilisées 9
4.1.4 Pertinence 9
4.2 Labyrinthes parfaits : Version TP 9
4.2.1 Algorithme utilisé 9
4.2.2 Pseudo-code 10
4.2.3 Structures de données utilisées 10
4.2.4 Pertinence 10
4.3 Labyrinthes parfaits : Version originale - Fusion 10
4.3.1 Algorithme utilisé 10
4.3.2 Pseudo-code 11
4.3.3 Structures de données utilisées 11
4.3.4 Pertinence 11
4.4 Labyrinthes aléatoires 12
4.4.1 Algorithme utilisé 12
4.4.2 Pseudo-code 12
4.4.3 Structures de données utilisées 13
4.4.4 Pertinence 13

5 Efficacité **14**
5.1 Méthode 14
5.1.1 Test de performance 14
5.1.2 Modélisation 15
5.2 Résultats 18

5.2.1 Labyrinthes aléatoires 18
5.2.2 Labyrinthes parfaits 18
5.3 Conclusion 18

6 Complément et Bilan **20**
6.1 Distance entrée / sortie 20
6.2 Placement des entités 20

7 Conclusion **21**

1 Introduction

Ce rapport présente la partie algorithmique du projet **SAÉ S3.02 - Labyrinthe**, une application avec génération de divers labyrinthes.

L'objectif est de décrire notre solution, dogmatisée par l'efficacité d'exécution et les demandes du sujet.

2 Investissement dans le projet

Groupe G3.3

- Victor Bredelle
- Antonin Marouze
- Romain Harlaut
- Angèl Zheng
- Baptiste Lavogiez

Nom	Victor	Antonin	Romain	Angèl	Baptiste	Total
Part	⟨Data 2.1⟩	⟨Data 2.2⟩	⟨Data 2.3⟩	⟨Data 2.4⟩	⟨Data 2.5⟩	100%

Table 2.1 – Répartition de l'investissement

1. Victor

2. Antonin

3. Romain

4. Angèl

5. Baptiste

> **Le groupe**

Information

L'ensemble des contributeurs a collaboré à l'élaboration du diagramme de classes.

3 Structures de données pour les labyrinthes

3.1 Labyrinthes aléatoires

3.2 Labyrinthes parfaits

4 Algorithmes de génération

4.1 Distance entrée / sortie

Cet algorithme sera utilisé dans les algorithmes à venir.

Le placement d'une distance entrée / sortie sera ici considéré comme un algorithme à part. Le respect strict de la distance nécessite en effet de devoir associer à chaque position du labyrinthe, depuis l'entrée, la distance les séparant.

4.1.1 Algorithme utilisé

Une approche par niveaux sera utilisée ici. L'algorithme sera un **parcours en largeur**.

4.1.2 Pseudo-code

On note :

- d : la distance cible entre l'entrée et la sortie.

pseudocode

```
créer une position aléatoire entrée dans le labyrinthe

créer une file vide
créer une map distances vide
créer une liste positionsResultat vide
ajouter entrée à la file
ajouter (entrée, 0) à distances
niveauActuel = 0
distanceReelle = 0

tant que la file n'est pas vide :
    tailleNiveau = taille de la file

    si niveauActuel > d :
        arrêter le parcours

    si niveauActuel = d :
        pour i de 0 à tailleNiveau :
            position = retirer premier élément de la file
            ajouter position à positionsResultat
            distanceReelle = niveauActuel
            arrêter le parcours

    pour i de 0 à tailleNiveau :
    positionCourante = retirer premier élément de la file
    pour chaque direction {haut, bas, gauche, droite} :
    prochaine = calculer position adjacente selon direction
    si prochaine valide et prochaine non visitée et pas de
       mur :
        ajouter (prochaine, niveauActuel + 1) à distances
        ajouter prochaine à la file
```

```
31
32      distanceReelle = niveauActuel
33      niveauActuel = niveauActuel + 1
34
35  si positionsResultat est vide :
36      pour chaque (position, distance) dans distances :
37      si distance = distanceReelle :
38      ajouter position à positionsResultat
39
40  sortie = choisir aléatoirement une position dans
       positionsResultat
41  définir entrée dans le labyrinthe
42  définir sortie dans le labyrinthe
43  mettre à jour la distance effective avec distanceReelle
```

Notre solution permet alors un strict respect de la distance entrée / sortie, et surtout, un calcul de la distance maximale possible afin de ramener la distance voulue à celle maximale si celle voulue était impossible dans ce labyrinthe. Cette notion est **importante pour la suite**.

La mise à jour de la distance effective permet, entre autres, de prévenir la structure de labyrinthes que la distance voulue a changé, permettant un retour utilisateur.

4.1.3 Structures de données utilisées

4.1.4 Pertinence

4.2 Labyrinthes parfaits : Version TP

4.2.1 Algorithme utilisé

4.2.2 Pseudo-code

pseudocode

```
toto titi
tant que toto
faire tata
```

4.2.3 Structures de données utilisées

4.2.4 Pertinence

4.3 Labyrinthes parfaits : Version originale - Fusion

4.3.1 Algorithme utilisé

4.3.2 Pseudo-code

pseudocode

```
toto titi
tant que toto
faire tata
```

4.3.3 Structures de données utilisées

4.3.4 Pertinence

4.4 Labyrinthes aléatoires

L'algorithme de génération de labyrinthe aléatoire héritant de l'algorithme parfait, cette sous-section est détaillée après la précédente.

L'objectif sera ici de partir d'un labyrinthe parfait et de le rendre imparfait. Cette solution permet un respect acquis de propriétés

Le chemin entre l'entrée et la sortie est, par définition, garanti, puisqu'un labyrinthe parfait est un graphe connexe.

La propriété connexe demeure inchangée tant qu'aucun mur n'est ajouté.

Cet algorithme n'ajoute aucun mur, il ne fait qu'un enlever.

Puisqu'un labyrinthe aléatoire est ici un graphe connexe, il existe toujours un chemin entre l'entrée et la sortie.

4.4.1 Algorithme utilisé

L'algorithme consiste à

Concernant la distance entrée / sortie, puisque sans paramètre dans le sujet, elle sera ramenée au maximum possible du labyrinthe, comme prévu par l'algorithme de placement.

4.4.2 Pseudo-code

On note :

- m : un labyrinthe parfait
- p : le pourcentage de murs souhaité

pseudocode

```
créer un fichier csv de nom "a_n_e_f"
tant que n < f :
    t1 = mesurer le temps actuel
    générer un labyrinthe d'algorithme (a) de taille n (
        largeur = n, hauteur = 2*n)
    t2 = mesurer le temps actuel
    écrire sur la ligne actuelle (n, t2 - t1)
    n = n + e
    passer à la ligne suivante
```

pseudocode

```
toto titi
tant que toto
faire tata
```

4.4.3 Structures de données utilisées

4.4.4 Pertinence

5 Efficacité

5.1 Méthode

La présente sous-section expose notre méthodologie de manière théorique, divisée en deux parties distinctes.

1. Test de performance
2. Modélisation

5.1.1 Test de performance

On note :

- a : l'algorithme du labyrinthe,
- n : la taille initiale du labyrinthe,
- e : l'écart entre deux tests successifs,
- f : la taille maximale (fin du test).

$$\forall n > 2,\ n \in \mathbb{N}, \forall e \in \mathbb{N} :$$

pseudocode

```
créer un fichier csv de nom "a_n_e_f"
tant que n < f :
    t1 = mesurer le temps actuel
    générer un labyrinthe d'algorithme (a) de taille n (
        largeur = n, hauteur = 2*n, distance = n)
    t2 = mesurer le temps actuel
    écrire sur la ligne actuelle (n, t2 - t1)
    n = n + e
    passer à la ligne suivante
```

Information

Lorsque la distance entrée est de 0, l'algorithme de distance entrée/sortie ne s'exécute pas. Cela permet d'observer le coût brut de la génération ou de la distance entrée/sortie, plutôt qu'une limitation à une combinaison des deux. Cette idée sera appliquée dans certaines analyses.

Taille (n)	Temps de génération (ms)
10	12
15	18
20	25
25	30
30	38
35	45
⋮	⋮
495	1200
500	1250

Table 5.1 – Exemple de sortie pour $a = \text{STANDARDLARGEUR}, n = 10, e = 5, f = 500$

Implémentation Java

5.1.2 Modélisation

Ces fichiers requièrent alors une analyse par un script concis en *matplotlib*.

python

```python
import os
import pandas as pd
import matplotlib.pyplot as plt
from PIL import Image
from io import BytesIO

d = "res/benchmarks"
files = [f for f in os.listdir(d) if f.endswith(".csv")]
for i,f in enumerate(files): print(i,f)
i = int(input("Choix: "))

```

```
df = pd.read_csv(os.path.join(d, files[i]))

plt.plot(df["taille"], df["temps(ms)"])
plt.xlabel("taille")
plt.ylabel("temps (ms)")
plt.title(files[i])
plt.tight_layout()

buf = BytesIO()
plt.savefig(buf, format="png")
plt.show()
plt.close()

buf.seek(0)
img = Image.open(buf)
os.makedirs(d+"/modelisations", exist_ok=True)
img.save(os.path.join(d+"/modelisations", files[i].replace(
    ".csv", ".png")))
```

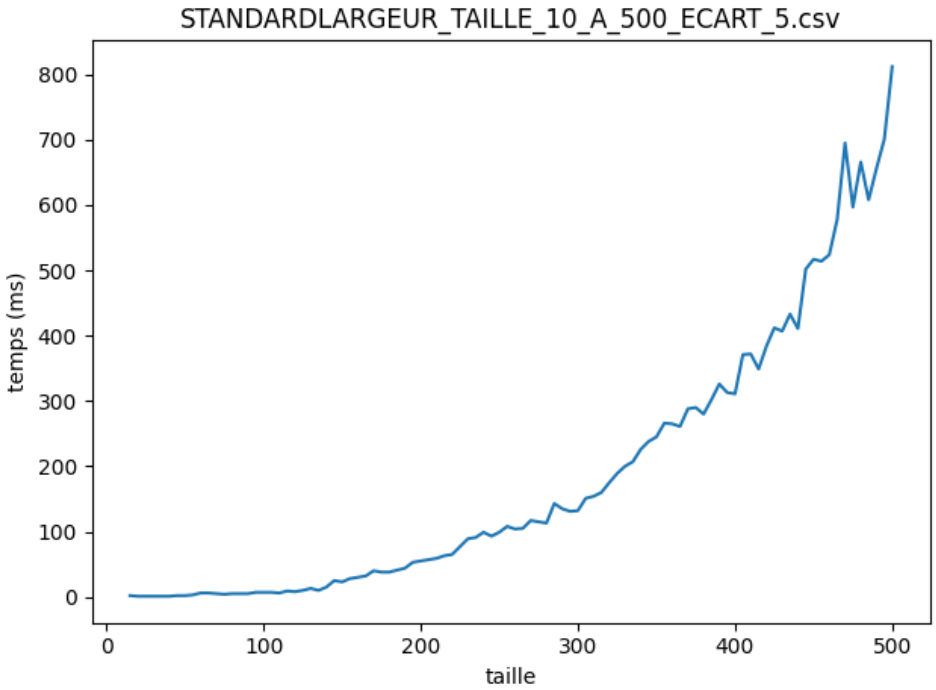

Figure 5.1 – Exemple de sortie pour $a = \text{STANDARDLARGEUR}, n = 10, e = 5, f = 500$

Seront uniquement présentés en résultat ces modélisations graphiques.

5.2 Résultats

5.2.1 Labyrinthes aléatoires

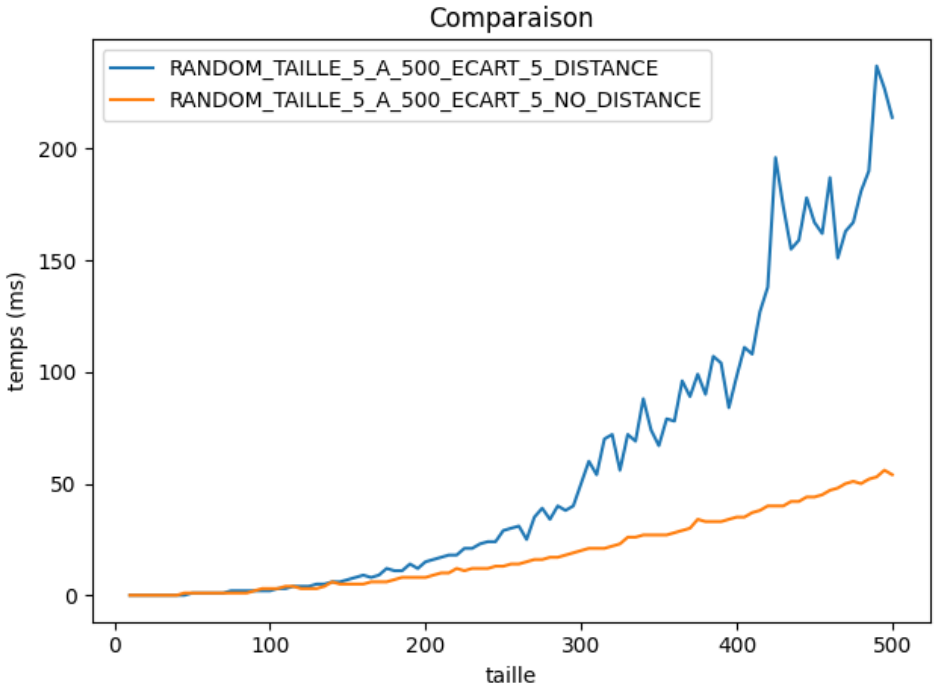

FIGURE 5.2 – $a = \text{RANDOM}, n = 5, e = 5, f = 500$

5.2.2 Labyrinthes parfaits

Algorithme vu en TP

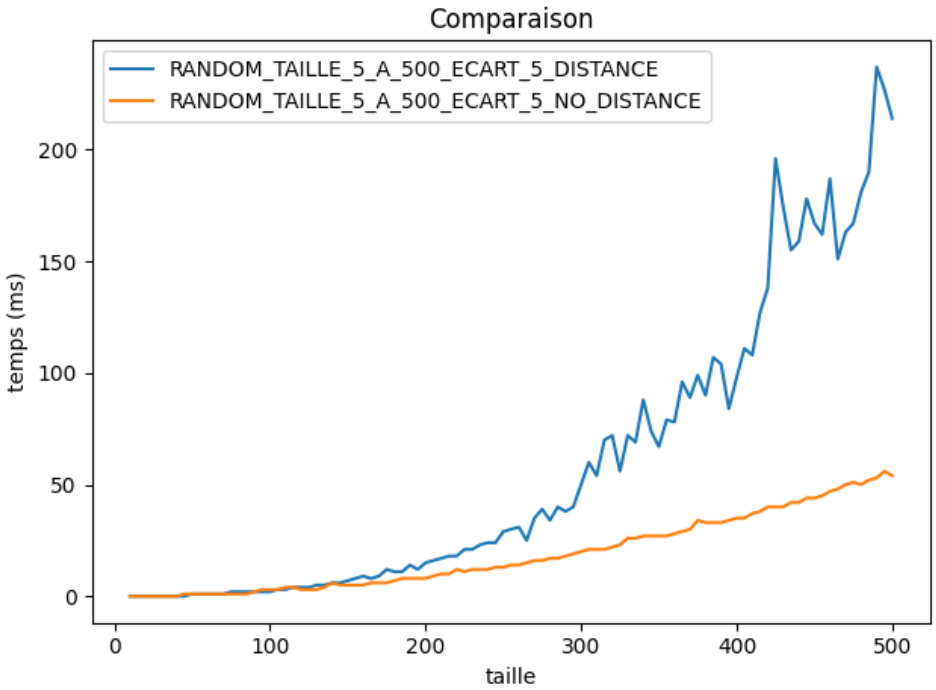

Figure 5.3 – $a = \text{PERFECT}, n = 5, e = 5, f = 500$

5.2.3 Comparaison parfaits / aléatoires

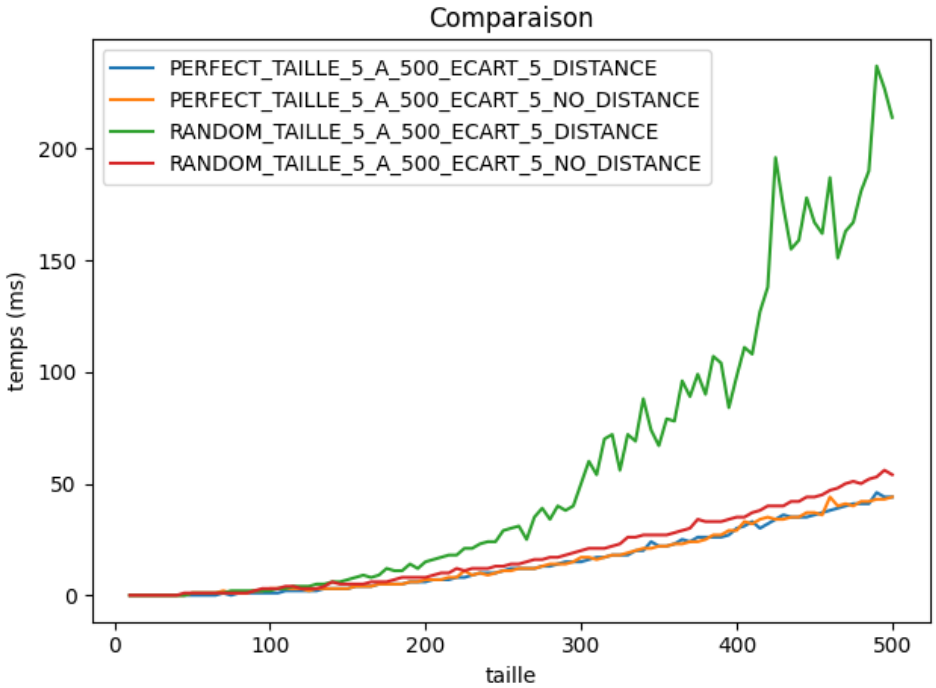

FIGURE 5.4 – Enter image caption...

Algorithme original : Fusion

5.3 Conclusion

6 Complément et Bilan

6.1 Distance entrée / sortie

Notre solution actuelle est optimisée, mais elle n'est apparue qu'après plusieurs tentatives. En effet, notre première solution consistait à :

- Faire un dictionnaire (clé : Position, valeur : Distance) de toutes les positions depuis la position en paramètre par parcours en largeur.
- Puis, choisir toutes les positions satisfaisant

En réalité, cette approche est inefficace puisqu'elle implique de parcourir absolument tout le labyrinthe. Elle avait été imaginée pour provisionner une solution simple au problème du maximum de la distance entrée / sortie.

Les performances de l'algorithme parfait ont été multipliées par 10 après application de la nouvelle solution.

6.2 Placement des entités

Notre projet comporte des **entités** (*joueurs, monstres, sorties...*) qui doivent être placées initialement selon des stratégies. Le problème étant très similaire à celui du placement de l'entrée et de la sortie, un algorithme très similaire a été implémenté pour le calcul de leurs positions de départ.

La différence étant que le placement vérifie l'absence d'entités sur cette case. L'algorithme enlève alors les positions occupées pour le niveau exploré. Si le niveau demandé ne comporte aucune position valide, alors

7 Conclusion